

Weekly Rapport — Weeks 10-12 (Final Sprint)

Real-Time Anomaly Detection Platform for Branch Teller Operations

Intern: Yassine — Amen Bank, Direction Centrale du Système d'Information

Period covered: July 9 – July 25, 2026

1. Executive Summary

This three-week sprint brought the project from a validated streaming pipeline with 10 services to a fully observable, self-correcting production system with 20 Docker Compose services. Seven major subsystems were designed and delivered: sink consumers with schema redesign, batch enrichment rewrite with Airflow orchestration, a four-page Streamlit supervisor dashboard, model retraining on the aligned 30-feature schema, a cold-start scoring guard, a complete MLOps subsystem (MLflow + drift-triggered retraining with three independent triggers), and production error handling (dead letter queue + structured JSON logging). The system is now feature-complete.

2. Session Timeline

Date	Focus	Key Deliverable
July 14	Sink consumers + schema redesign	Raw-events sink, decisions sink, PostgreSQL schema rebuild
July 16	Batch enrichment + Airflow	EnrichmentCore purity, 2 validated DAGs, 17 Docker services
July 21	Streamlit dashboard	4-page dashboard, feedback endpoint, Docker integration
July 23	Model retraining + pipeline validation	AE 0.9714 / LSTM 0.9355, cold-start problem identified
July 24 (AM)	Cold-start guard + buffer fix	Neutral scoring for new clients, pipeline validated clean
July 24 (PM)	MLflow + drift-triggered retraining	3-trigger retrain subsystem, 20 Docker services
July 25	DLQ + structured logging	ResilientConsumer, JSON logging across streaming layer

3. Sink Consumers and Schema Redesign (July 14)

3.1 Problem Statement

The Streamlit dashboard requires data from PostgreSQL, but Streamlit's script re-run model is incompatible with Kafka's consumer group protocol. Long-lived Kafka consumers cannot coexist with Streamlit's execution model, which re-executes the entire script on every user interaction.

3.2 Solution: Decoupled Sink Pattern

Two new long-running consumers were added to the streaming pipeline, each with its own consumer group (independent from the transform-and-forward consumers):

Consumer	Input Topic	Writes To	Prometheus Port
Raw-Events Sink	<code>raw-events</code>	PostgreSQL <code>transactions</code>	9103
Decisions Sink	<code>decisions</code>	PostgreSQL <code>alerts</code> + Redis risk history	9104

Both use `ON CONFLICT (event_id) DO NOTHING` for idempotency and the same `ServiceMetrics` pattern for Prometheus observability.

The decisions sink performs two write paths per event: (A) INSERT into `alerts` with tier, fused score, SHAP explanation, and supervisor decision fields, and (B) Redis risk history update — incrementing `alert_{tier}_30d_count` for REVIEW/ALERT/BLOCK events while skipping INFO events.

3.3 Schema Redesign

Three schema changes were required to support the sink architecture:

The `transactions` table had oracle label columns (`is_anomaly`, `anomaly_type`) removed — ground-truth labels should never exist in a production transactions table. A new `evaluation_labels` table was created with a safe FK to `transactions` (batch-only context, no race condition risk), storing `is_anomaly` and `anomaly_type` for model evaluation. The `alerts` table was rebuilt with `alert_tier` as a PostgreSQL ENUM, plus `branch_id`, `amount`, `operation_type`, `regulatory_flags` (JSONB), `explanation` (JSONB with `reasons` and `raw` sub-keys), and supervisor decision fields.

Critical architectural decision: The FK from `alerts` to `transactions` was deliberately dropped. In the streaming path, the raw-events sink and decisions sink consume from different topics in parallel. A FK would create a race condition — the decisions sink may arrive before the raw-events sink inserts the transaction row. Referential integrity is guaranteed by pipeline topology, not database constraints. This is intentional, not an oversight.

4. Batch Enrichment Rewrite and Airflow (July 16)

4.1 Feature Alignment as Hard Dependency

The streaming pipeline's enrichment uses a 30-feature contrast design, but the batch enrichment script (`batch_enrichment.py`) was still computing 74 features from the old schema. This mismatch meant models trained on batch-computed features produced systematically elevated anomaly scores in streaming inference. The batch rewrite was prioritized over the dashboard precisely because misaligned features made the dashboard meaningless.

4.2 EnrichmentCore Purity

The last Redis call was removed from `_compute_features()` by hoisting the archetype baseline fetch into `enrich_event()`. This made `_compute_features()` a pure function: same inputs always produce same outputs, no side effects, no I/O. The archetype baseline is now passed as a parameter. This guarantees that streaming and batch use identical logic with zero drift risk — the single most important invariant in the system.

4.3 Batch Enrichment Rewrite

The rewritten `batch_enrichment.py` calls `_compute_features()` directly from `EnrichmentCore`, using a server-side PostgreSQL cursor with `CHUNK_SIZE=5000` to avoid OOM kills (the previous version loaded all 260k profile snapshots into memory). The `recent_events_buffer` (list of 50 events per client) is stripped at load time since batch enrichment doesn't need it. All three batch scripts received `ON CONFLICT` upserts for Airflow retry safety — any task can be retried without data corruption.

4.4 Airflow Integration

Apache Airflow was integrated using LocalExecutor with shared PostgreSQL metadata. A lightweight custom Docker image (`Dockerfile.airflow`) with `requirements-airflow.txt` containing only five dependencies — no PyTorch) keeps the image small.

Two DAGs were validated end-to-end:

`nightly_profiles` — 8 tasks with validation gates: truncate staging tables → rebuild client profiles → rebuild employee profiles → rebuild archetype baselines → validate row counts → hydrate Redis → validate Redis keys → mark complete. Each task has explicit dependencies and the validation gates prevent downstream tasks from running on corrupt data.

`weekly_retrain` — 3 tasks: export training features → retrain models → validate metrics. The retrain task was initially a placeholder, later wired with DockerOperator (see Section 8).

Docker Compose grew from 12 to 17 defined services (16 running — `airflow-init` exits after initialization). Issues resolved included `DB_HOST` vs `POSTGRES_HOST` environment variable naming, shared log volume + secret key for webserver-scheduler communication, and `amen_bank` vs `amen_anomaly` database name mismatch.

5. Streamlit Supervisor Dashboard (July 21)

5.1 Architecture Decisions

Four architecture decisions were made before any code was written:

Navigation: Multi-page structure (one `.py` per page) chosen over single-page for lower latency — each page loads only its own data and dependencies.

Data access: Reads come directly from PostgreSQL with `st.cache_data` caching. Writes go through FastAPI endpoints only — the dashboard never writes to the database directly. This preserves the single-writer principle and enables audit logging on the API side.

Connection resilience: A `safe_query()` wrapper in `db.py` automatically reconnects on stale PostgreSQL connections, which occur after Docker restarts or idle timeouts.

Real-time refresh: `streamlit-autorefresh` at 10-second intervals rather than manual refresh or WebSocket. Streamlit does not support native WebSocket push; the auto-refresh component provides 90% of the UX with minimal implementation cost.

5.2 Dashboard Pages

Alerts page — Filterable list of alerts with expandable detail for each alert. The detail view displays SHAP explanations using `explanation.reasons` for the NLG narrative (human-readable sentences generated by deterministic templates) and `explanation.raw` for feature-level detail (contribution values per feature). Confirm/reject buttons allow supervisors to provide feedback.

KPIs page — Metric cards showing total alerts, pending reviews, confirmation rate, and average score. Four Plotly charts: alerts over time, distribution by tier, distribution by operation type, and top clients by alert count.

Simulation page — Form with dropdowns populated from known clients/employees in the alerts table, POSTs to the FastAPI `/api/v1/score` endpoint. Returns the full scoring output including tier, fused score, SHAP explanation, and rule triggers.

Health page — Six service status indicators (enrichment, scoring, decision, raw-events sink, decisions sink, simulation API), throughput rates, p95 latency, error counts by class, and consumer lag per topic/partition from the Kafka exporter.

A shared `components.py` module renders the sidebar on every page, persisting pipeline status and pending alert counts across navigation.

5.3 Feedback Endpoint

A new `POST /api/v1/feedback` endpoint was added to the FastAPI simulation API. The `FeedbackRequest` Pydantic model uses `event_id` (not `alert_id`) to match the database schema. The endpoint updates `supervisor_decision` and `decision_timestamp` in the `alerts` table and upserts into `evaluation_labels` for the retraining feedback loop.

Synchronous `psycpg2` was chosen over `asynpg` given the low-frequency nature of supervisor actions and the synchronous codebase.

6. Model Retraining on 30-Feature Schema (July 23)

6.1 Retraining Pipeline

The retraining pipeline runs `batch_enrichment.py` to generate `data/training_features.csv` using the same `_compute_features()` function used in streaming — this is the critical guarantee of zero feature drift. A unified `src/training/train_all.py` calls `prepare()` once then trains both models sequentially, also computing and saving reference score distributions (`ae_ref_scores.npy`, `lstm_ref_scores.npy`) used for percentile-based score normalization.

6.2 Results

Model	Old AUC (74 features)	New AUC (30 features)
Autoencoder	0.9853	0.9714
LSTM	0.9866	0.9355

The AUC drop was expected. The Autoencoder's performance held well because reconstruction error is less sensitive to input dimensionality. The LSTM lost more because sequence modeling benefits from richer per-step representations — each time step now carries 30 features instead of 74, reducing the information available for temporal pattern learning.

Framing for rapport and soutenance: These AUCs validate the system's ability to learn and serve predictions end-to-end on synthetic data, not a production benchmark. The engineering argument: the system is ready for real data — plug in real transactions, retrain, and the pipeline handles the rest without code changes.

6.3 Airflow Weekly Retrain Wired

The `weekly_retrain` DAG's placeholder task was replaced with a `DockerOperator` that runs the training image (`amen-anomaly-training:latest`). Configuration: Docker socket mounted into Airflow containers, `force_pull=False` (uses locally built image), `mount_tmp_dir=False` (avoids Airflow creating temp directories inside the container).

7. Cold-Start Guard and Buffer Fix (July 24 AM)

7.1 Problem

Zero-velocity feature vectors (clients with empty or near-empty event buffers) fall far outside the training distribution. The training pipeline applied a 21-day warmup filter, so models never learned what "no history" looks like. Result: cold-start clients received anomaly scores of ~0.999, saturating into ALERT/BLOCK tiers.

7.2 Solution: Three-Tier Scoring Progression

Buffer Length	Behavior
0-4 events	Neutral score 0.5, <code>cold_start=True</code> , <code>w_lstm=0.0</code> , tier capped at REVIEW
5-9 events	AE-only scoring (LSTM weight = 0), full tier range
10+ events	Full AE+LSTM fusion with sigmoid weighting by account maturity

In `ScoringCore`, buffer length is checked via Redis. When `buffer_length < 5`, a neutral response is returned with `fused_score: 0.5`, `cold_start: True`, and no SHAP explanations. In `DecisionCore`, cold-start events still pass through all rules (regulatory triggers must still fire) but the final tier is capped at REVIEW maximum using `AlertTier` IntEnum comparison.

7.3 Buffer Duplication Fix

Root cause: `hydrate_redis.py` seeds sequence buffers from `recent_events_buffer` in profile snapshots, but the simulator then replays the same events — doubling velocity

features (`tx_count_24h`, `cumulative_amount_24h`). Fix: added `--skip-buffers` flag to `hydrate_redis.py` via argparse, updated the docker-compose command. The simulator received a `start_date` parameter for temporally aligned replay.

7.4 Validated Pipeline Distribution

After fixes: 89% INFO / 11% REVIEW / 0.35% ALERT, score range 0.015-0.999. Cold-start events appear correctly in the database with `fused_score: 0.5` and tier capped at REVIEW.

Demo configuration: Use `--skip-buffers` on hydrate-redis + `start_date='2025-06-20'` on the simulator.

8. MLflow and Drift-Triggered Retraining (July 24 PM)

8.1 MLflow Integration

MLflow server deployed with flat-file backend (`mlruns/` volume, port 5001) — a database backend was not justified for the project's scale and timeline. `train_all.py` was updated to log two independent runs (one AE, one LSTM) under the experiment `amen-anomaly-detection`, each logging parameters, metrics, and artifacts.

A promotion gate reads `models/production_metrics.json` and only saves new model `.pt` files if the new AUC meets or exceeds the production AUC (`>=` comparison — prefer fresh models at equal performance). First run bootstraps the file. Validation: AE 0.9663 promoted, LSTM 0.9340 promoted.

MLflow's role is post-mortem debugging: when the promotion gate rejects a model at 4 AM, engineers inspect the run on Monday. It is a passive logging tool, not an automation engine.

8.2 Drift Detection Chain

A four-component chain detects score distribution drift and triggers retraining automatically:

Prometheus alert rule (`config/alert_rules.yml`): monitors the fraction of events with fused scores above the T1 threshold (0.95). If more than 15% of events exceed T1 for 1 continuous hour, the `ScoreDistributionDrift` alert fires.

AlertManager (`prom/alertmanager:v0.27.0`, port 9093): receives the firing alert, routes it to the webhook relay. `repeat_interval: 24h` prevents alert storms.

Webhook relay (`src/webhook/relay.py`, `Dockerfile.webhook` using `python:3.11-slim` ~150MB): a 20-line FastAPI app that receives the AlertManager POST and calls the Airflow REST API to trigger the `weekly_retrain` DAG with `conf={"trigger": "drift_alert"}`. The relay is internal-only (no exposed port). A lightweight image was chosen over the 2GB PyTorch training image.

Airflow REST API (enabled via `AIRFLOW__API__AUTH_BACKENDS=airflow.api.auth.backend.basic_auth`): receives the trigger and queues the DAG run. Full chain validated end-to-end with 200 response and DAG state `queued`.

8.3 Feedback Divergence DAG

A separate DAG (`dags/feedback_monitor.py`) runs daily, querying PostgreSQL for the supervisor rejection rate on ALERT+BLOCK decisions over a 7-day sliding window. If the rejection rate exceeds 50% with a minimum of 10 reviewed decisions, the DAG triggers retraining. The 10-review minimum prevents statistical noise from triggering retraining (at ~5 ALERT/BLOCK events per day with a 0.35% tier rate, 10 reviews accumulate in 2 days).

8.4 Three Retrain Triggers

Trigger	Mechanism	Frequency
Scheduled	Airflow cron	Weekly
Drift	Prometheus → AlertManager → Webhook → Airflow	On detection
Feedback	Airflow DAG → SQL check → conditional trigger	Daily check

Docker Compose grew from 17 to 20 services (+mlflow, alertmanager, webhook-relay).

9. Dead Letter Queue (July 25)

9.1 Problem

Prior to this change, a failed event in any streaming service was either silently dropped (lost data, no audit trail) or caused the consumer to stop (one bad event blocks everything behind it). Neither is acceptable in production.

9.2 Failure Classification

Two failure classes are distinguished:

Poison pills — malformed events, missing fields, schema violations, `ValueError`, `KeyError`. The event will never succeed no matter how many times it is retried. These are routed to the DLQ immediately with `retry_count=0`.

Transient failures — `redis.ConnectionError`, `redis.TimeoutError`, `psycpg2.OperationalError`, `ConnectionError`, `TimeoutError`, `OSError`. The event is valid but a dependency is temporarily unavailable. These are retried with exponential backoff (1s, 2s, 4s) before being routed to the DLQ with `retry_count=3`.

Classification uses `isinstance(e, TRANSIENT_ERRORS)` against a tuple of known transient exception types — explicit, easy to extend.

9.3 ResilientConsumer

A reusable `ResilientConsumer` class (`src/streaming/dlq.py`) wraps the processing function of each streaming service. Each wrapper provides its service name, source topic, processing function, and Kafka producer at construction time. The `handle(event)` method returns the result on success or `None` on failure (with the failed event routed to the `dlq` topic automatically).

The DLQ envelope contains: `original_event` (untouched), `source_topic`, `service`, `error_type` (`poison_pill` or `transient`), `error_message`, `stacktrace`, `retry_count`, `max_retries`, `first_failure_at`, `last_failure_at`.

All three streaming wrappers (enrichment, scoring, decision) were updated to use `ResilientConsumer` — each wrapper's try/except block was replaced with a single `resilient.handle(event)` call.

Fallback: If RedPanda itself is down and the DLQ send fails, the error is printed to stderr as a last resort. In production, this would write to a local file — noted as a known limitation.

10. Structured JSON Logging (July 25)

10.1 Problem

All streaming services used `print()` statements. With 20 Docker containers producing unstructured output, debugging requires manually scanning `docker logs` across multiple containers with no consistent format to search or filter on.

10.2 Solution

A `JsonFormatter` class (`src/streaming/logger.py`) extends Python's `logging.Formatter` to output single-line JSON objects. Each log line contains: `timestamp` (UTC ISO), `service`, `level`, `message`, plus optional contextual fields (`event_id`, `client_id`, `employee_id`, `source_topic`, `error_type`, `retry_count`, `score`, `alert_tier`) passed via Python's `extra` parameter.

A `setup_logger(service_name)` factory function creates a logger with the JSON formatter attached to stdout. Idempotency guard prevents duplicate handlers if called twice. `propagate=False` prevents duplication from the root logger.

All `print()` calls in the streaming wrappers and DLQ module were replaced with structured `logger.info()`, `logger.warning()`, and `logger.error()` calls with appropriate `extra` fields.

10.3 Validated Output

```
json
{"timestamp": "2026-07-25T10:12:19Z", "service": "enrichment", "level": "INFO", "message": "enrichment service started"}
{"timestamp": "2026-07-25T10:13:22Z", "service": "dlq", "level": "WARNING", "message": "dlq service started"}
{"timestamp": "2026-07-25T10:13:23Z", "service": "dlq", "level": "INFO", "message": "dlq service started"}
{"timestamp": "2026-07-25T10:13:23Z", "service": "enrichment", "level": "WARNING", "message": "enrichment service started"}
```

11. Docker Compose Service Growth

Milestone	Service Count	New Services
End of Weeks 8-9	10	— (baseline)
July 14	12	raw-events-sink, decisions-sink
July 16	17	airflow-init, airflow-scheduler, airflow-webserver, kafka-exporter, simulation-api
July 24	20	mlflow, alertmanager, webhook-relay

12. Key Architectural Principles Reinforced

Feature alignment is a hard dependency. Training and inference must use identical feature computation paths. The batch enrichment rewrite was prioritized over the dashboard because misaligned features (74 vs 30) produced systematically elevated anomaly scores that made the dashboard meaningless.

EnrichmentCore purity. `_compute_features()` must be a pure function with no I/O. The archetype baseline is passed as a parameter. This guarantees streaming and batch use identical logic with zero drift risk.

Rules can only escalate, never de-escalate. Cold-start caps the tier at REVIEW, but regulatory rules still fire and are logged for audit regardless.

Health monitoring failure domain separation. Health reporting must not share a failure domain with the monitored service. The DLQ's last-resort stderr fallback is the same principle — if the error handling infrastructure is down, you fall back to the most primitive channel available.

Alerts→transactions FK deliberately dropped. Race condition between parallel sink consumers. Intentional architecture, not an oversight.

13. Bugs Resolved

Bug	Root Cause	Fix
<code>KeyError: 'w_lstm'</code> in cold-start output	Missing field in <code>_build_cold_start_output</code>	Added <code>w_lstm: 0.0</code> to output dict
Buffer duplication → inflated scores	<code>hydrate_redis</code> seeds buffers, then simulator replays same events	<code>--skip-buffers</code> flag on hydrate-redis
Simulation API DB connection failure	Missing <code>DB_HOST</code> in docker-compose env	Added all 5 DB env variables
Corrupted <code>config/prometheus.yml</code>	File contained docker-compose content	Restored proper scrape configuration
Sidebar disappearing on page navigation	Streamlit re-run model clears non-page state	Shared <code>components.py</code> renders sidebar on every page
<code>production_metrics.json</code> bootstrap	First run has no file, comparison fails	Bootstrapped with current AUCs, first run uses <code>>= 0.0</code>
OOM kills in batch enrichment	Loading 260k profiles with 50-event buffers each	Server-side cursor + buffer stripping at load

14. System Status at End of Sprint

Services: 20 Docker Compose services, all validated.

Models: AE AUC 0.9714, LSTM AUC 0.9355 (30-feature schema).

Pipeline distribution: 89% INFO / 11% REVIEW / 0.35% ALERT.

Retraining: 3 independent triggers (scheduled, drift, feedback) with MLflow tracking and promotion gate.

Error handling: Dead letter queue with exponential backoff retry, structured JSON logging across streaming layer.

Dashboard: 4 pages (Alerts, KPIs, Simulation, Health) with real-time refresh and supervisor feedback loop.

15. Remaining Work

Item	Type	Scope
Rapport de stage (30-40 pages)	Document	Assembled from weekly reports + LaTeX checkpoints
Soutenance preparation	Document + Demo	Slides, demo script, live walkthrough
Kubernetes / horizontal scaling	Document-only	Production roadmap in rapport
CI/CD pipelines	Document-only	Production roadmap in rapport
Secrets management	Document-only	Production roadmap in rapport
Load testing	Document-only	Production roadmap in rapport
Data retention policies	Document-only	Production roadmap in rapport

Deadline: July 28, 2026.